ERODE SENGUNTHAR
ENGINEERING COLLEGE
AUTONOMOUS

PERUNDURAI, ERODE - 638 057

(Approved by AICTE, New Delhi, Permanently Affiliated to Anna University, Chennai& Accredited by National Board of Accreditation (NBA), New Delhi, National Accreditation Assessment Council, Bangalore& IE(I), Kolkata)

BONAFIDE CERTIFICATE

Register No. .....

Certified that this is the Bonafide Record of Work Done

Name of the Student: _____

Branch : Information Technology

Name of the Lab : Mobile Application Development (23IT703)

Year/Semester : IV / VII

Faculty Incharge Dean / IT

Submitted for the End Semester Practical Examination held on .....

Internal Examiner External Examiner

INSTITUTE VISION MISSION

VISION

- Vision of Erode Sengunthar Engineering College is to become a World Class Technical Institution and Scientific Research Centre for the Benefit of the Society.

MISSION

Erode Sengunthar Engineering College will strive continuously to

- Create Positive difference to Society through Innovative Teaching – Learning Process.
- Impart Value Based Technical Education to the Students from across various Socio-Economic backgrounds.
- Build State of art infrastructure for high quality Research and Development capabilities on par with the finest in the Globe and widen student's horizons beyond Class Room.
- Bring out Competent, Ethically Strong and Quality Professionals.

DEPARTMENT VISION MISSION

VISION

To emerge as a Centre of Excellence with an open growth to foster a groomed, professionally competent and socially responsible IT intellectual graduates in facing dynamic and diversified unseen challenges in the technical advancement of Information and Communication Sphere.

MISSION

1. To enhance the problem-solving skills of the students to build their excellence in providing hardware, software-based solutions in Information Technology arena.
2. To promote students with tools to become a prolific, responsible global citizens and lifelong learners
3. To create an inquisitiveness in research with a need based quality training.
4. To develop highly skilled, vibrant, and exceptionally talented graduates to the industries with a strong ethical foundation.

PROGRAM EDUCATIONAL OBJECTIVE (PEO)

PEO1: Preparation: To be skilled in the usage of modernized tools for solving critical and sensitive Information Technology and Communication Challenges through creativity and durability.

PEO2: Core Competence and Professionalism: To be able to impart extensive range of Information Technology related problems in order to tailor the essentials of Industry, Entrepreneurship and Research.

PEO3: Breadth: To nurture with an effectual leadership, good managerial and soft skills among multi-disciplinary teams thereby making a positive vibration on the global environment and society

PEO4: Learning Environment: To explore systematic decision making and problem solving abilities by applying innovative and sustainable principles to the real time challenges through moral ethics and professional approach.

PROGRAM OUTCOMES (POs)

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO1: (Statistical Base Knowledge): Design and develop algorithms and applications by applying mathematical knowledge and systematic principles in the field of Information Technology to meet industrial needs.

PSO2: (Advanced Technical Skills): Effective and in-depth knowledge of latest developments such as Artificial Intelligence, Data Science, Machine Learning, Web Technologies, Cyber security and IOT etc. in the domain of Computer Communication Networks.

PSO3: (Personality Development): Comprehend suitable current practices followed in Information and Communication Engineering by incorporating ethical principles into the user environment and expose Teams Sprit and Leadership Qualities.

INDEX

Ex.No	Date	Name of the Experiment	Page No	Marks	Signature
1.		Study and installation of Flutter / Kotlin multi-platform environment			
2.		Develop an application that uses Widgets, GUI components, Fonts, and Colors.			
3.		Develop a native calculator application.			
4.		Develop a gaming application that uses 2-D animations and gestures.			
5.		Develop a movie rating application (similar to IMDB)			
6.		Develop an application to connect to a web service and to retrieve data with HTTP.			
7.		Develop a simple shopping application.			
8.		Design a web server supporting push notifications			
9.		Develop an application by integrating Google maps			
10.		Mini Project- BMI Calculator App			
11.		**Content Beyond Syllabus** Real-Time on-Device Object Detection Using Flutter and Tensor flow Lite			

Average :

Ex No:1 Date:	STUDY AND INSTALLATION OF FLUTTER/KOTLIN MULTI-PLATFORM ENVIRONMENT

Aim:

To Study and install Flutter/Kotlin multi-platform environment

Procedure:

Flutter:

Install Flutter SDK:

Download the Flutter SDK from the official Flutter website. Extract the downloaded archive and add the flutter/bin directory to your system PATH.

Set up an IDE (Integrated Development Environment):

You can use Android Studio, IntelliJ IDEA, or Visual Studio Code for Flutter development. Install the Flutter and Dart plugins for your chosen IDE.

Run flutter doctor:

Open a terminal and run flutter doctor to check for any dependencies that need to be installed. Follow the instructions provided by flutter doctor to resolve any issues.

Create a Flutter Project:

Run the following commands in your terminal:

flutter create my_flutter_project

cd my_flutter_project

Run the App:

Connect a device or start an emulator and run flutter run in the project directory.

Kotlin:

Install Kotlin:

You can install Kotlin by following the instructions on the official Kotlin website.

Set up an IDE:

Kotlin Multiplatform projects are well-supported in IntelliJ IDEA. Install the Kotlin plugin for your IDE.
Create a Kotlin Multiplatform Project:

Create a new Kotlin Multiplatform project using a template or set it up manually.

Configure Shared Code:

Define the common code that will be shared across platforms. This code will be written in Kotlin and can include business logic, data models, etc.

Platform-Specific Code:

Implement platform-specific code for Android and iOS. This involves creating separate modules or directories for each platform and writing the necessary code in Kotlin (for common logic) and Swift/Kotlin (for platform-specific code).

Build and Run:

Build the project using the appropriate build command for your project structure (e.g., Gradle for Android). Run the app on Android and iOS devices/emulators Remember that integrating Flutter and Kotlin Multiplatform in the same project can be complex, and it may require additional setup and configuration. Ensure that you carefully follow the documentation for both technologies and understand the integration points.

EXPERIMENT(60 Marks)		
Description	**Max Marks**	**Awarded**
Observation	**50**	
Record	**10**	
Total	**60**	

Result:

Thus the study and installation of Flutter/Kotlin multi-platform environment has been executed successfully.

Ex No:2 Date:	DEVELOP AN APPLICATION THAT USES WIDGETS, GUI COMPONENTS, FONTS, AND COLORS.

Aim:

Develop an application that uses Widgets, GUI components, Fonts, and Colors.

Procedure:

1) Open your preferred Kotlin IDE.

2) Create a new Kotlin project.

3) Download the JavaFX SDK from the official website.

4) Extract the downloaded archive to a location on your computer.

5) In your Kotlin project, add the JavaFX library to your project's dependencies.

6) Create a new Kotlin file for your main application class, e.g., MyApp.kt.

7) Copy and paste the Kotlin code provided in the previous response into this file.

8) Open the Run/Debug Configuration in your IDE.

9) Add the following VM options:

--module-path/path/to/javafx-sdk-17/lib --add-modules javafx.controls,javafx.fxml Replace/path/to/javafx-sdk-17 with the actual path to your JavaFX SDK. Run the main function in your MyApp.kt file.

Program:

```
import javafx.application.Application

import javafx.geometry.Insets

import javafx.scene.Scene

import javafx.scene.control.Button

import javafx.scene.control.ComboBox

import javafx.scene.control.Label
```

```
import javafx.scene.control.TextField

import javafx.scene.layout.VBox

import javafx.scene.text.Font

import javafx.stage.Stage
class MyApp: Application() {

override fun start(primaryStage: Stage) {

primaryStage.title = "Widget App"

val root = VBox(10.0)

root.padding = Insets(10.0)

// Set a custom font

val customFont = Font("Arial", 12.0)

// Set background color

root.style = "-fx-background-color: #f0f0f0;"

// Create a label

val label = Label("Welcome to Widget App")

label.font = customFont

root.children.add(label)

// Create a button

val button = Button("Click Me!")

button.font = customFont

button.setOnAction {onButtonClick()}

root.children.add(button)
```

```
// Create a text field

val textField = TextField()

textField.font = customFont

root.children.add(textField)

// Create a combo box

val comboBoxValues = listOf("Option 1", "Option 2", "Option 3")

val comboBox = ComboBox<String>()

comboBox.items.addAll(comboBoxValues)
comboBox.font = customFont

root.children.add(comboBox)

val scene = Scene(root, 300.0, 200.0)

primaryStage.scene = scene

primaryStage.show()

}

private fun onButtonClick() {

println("Button Clicked!")

}

}

fun main() {

Application.launch (MyApp::class.java)

}
```

Output:



EXPERIMENT(60 Marks)		
Description	**Max Marks**	**Awarded**
Observation	**50**	
Record	**10**	
Total	**60**	

Result:

Thus the application that uses widgets, GUI components, Fonts and Colors has been executed successfully.

Ex No:3 Date:	DEVELOP A NATIVE CALCULATOR APPLICATION

Aim:

To develop a native calculator application.

Procedure:

Create a new Android Studio Project:

Open Android Studio and create a new project.

Choose "Empty Activity" template.

Design the UI:

Open res/layout/activity_main.xml and replace its content with the following Xml code

Define Button IDs and Operators:

Open MainActivity.kt and replace its content with the following kotlin code

Run your app:

Connect your Android device or start an emulator.

Click on the "Run" button in Android Studio.

Program:

Xml:

```
<?xml version="1.0" encoding="utf-8"?>

<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"

xmlns:tools="http://schemas.android.com/tools"

android:layout_width="match_parent"

android:layout_height="match_parent"
```

```
tools:context=".MainActivity">

<EditText

android:id="@+id/inputText"
android:layout_width="match_parent"

android:layout_height="wrap_content"

android:layout_marginTop="16dp"

android:layout_marginBottom="16dp"

android:layout_alignParentTop="true"

android:layout_centerHorizontal="true"

android:hint="Enter expression"

android:inputType="none"

android:textAlignment="center" />

<GridLayout

android:id="@+id/gridLayout"

android:layout_width="match_parent"

android:layout_height="wrap_content"

android:layout_below="@id/inputText"

android:layout_marginTop="16dp"

android:columnCount="4"

android:orientation="horizontal">

<!-- Buttons for digits and operators go here -->

</GridLayout>

</RelativeLayout>
```

Kotlin:

```
import android.os.Bundle

import android.view.View

import android.widget.Button

import android.widget.EditText

import androidx.appcompat.app.AppCompatActivity
import net.objecthunter.exp4j.ExpressionBuilder

class MainActivity: AppCompatActivity() {

private lateinit var inputText: EditText

override fun onCreate(savedInstanceState: Bundle?) {

super.onCreate(savedInstanceState)

setContentView(R.layout.activity_main)

inputText = findViewById(R.id.inputText)

val buttons = intArrayOf(

R.id.btno, R.id.btn1, R.id.btn2, R.id.btn3, R.id.btn4, R.id.btn5, R.id.btn6, R.id.btn7, R.id.btn8,
R.id.btn9, R.id.btnAdd, R.id.btnSub, R.id.btnMul, R.id.btnDiv, R.id.btnDot, R.id.btnEquals )

for (buttonld in buttons) {

val button = findViewById<Button>(buttonld)

button.setOnClickListener {onButtonClick(it)}

}

}

}

private fun onButtonClick(view: View) {
```

```
val currentText = inputText.text.toString()

val buttonText = (view as Button).text.toString()

when (view.id) {

R.id.btnEquals -> evaluate Expression(currentText)

R.id.btnDot -> handleDotButtonClick(currentText)

else -> inputText.setText("$currentText$buttonText")
}

private fun evaluateExpression (expression: String) {

try {

val result = Expression Builder (expression).build().evaluate()

inputText.setText(result.toString())

} catch (e: Exception) {

inputText.setText("Error")

}

} private fun handleDotButtonClick(currentText: String) {

val lastOperatorIndex = getLastOperatorIndex(currentText)

val lastDotIndex = currentText.lastIndexOf(".")

if (lastDotIndex > lastOperatorIndex) {

// If there is already a dot after the last operator, do nothing.

return

} inputText.setText("$currentText.")

} private fun getLastOperatorIndex(text: String): Int {
```

```
val operators = charArrayOf('+', '-', '*', '/')

var lastOperatorIndex = -1

for (operator in operators) {

val index = text.lastIndexOf(operator)

if (index > lastOperatorIndex) {

lastOperatorIndex = index

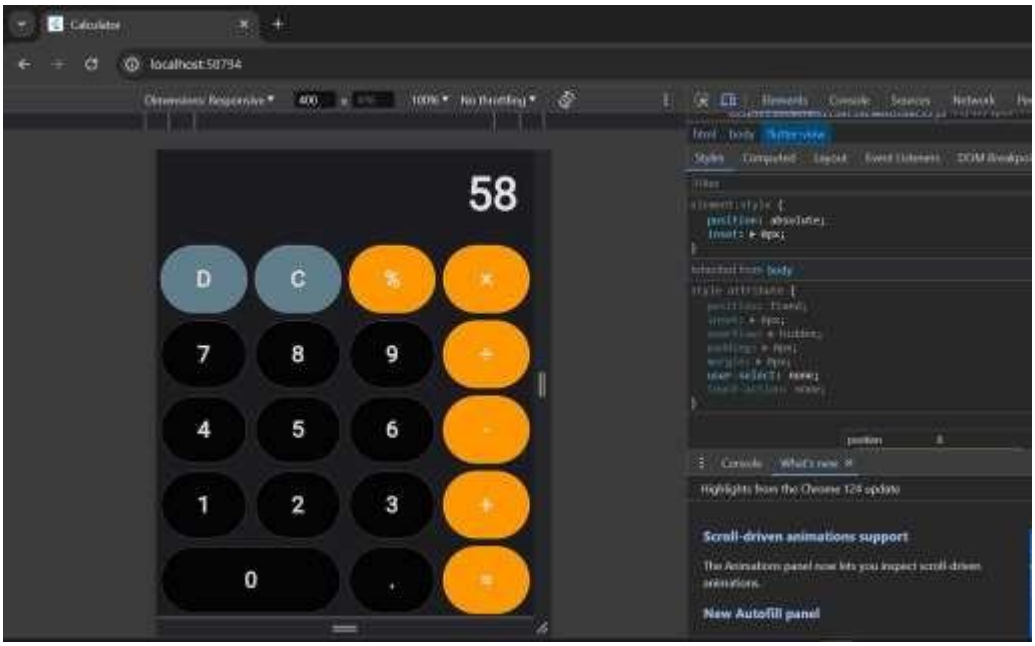
}

} return lastOperatorIndex

}

}
```

Output:



EXPERIMENT(60 Marks)		
Description	**Max Marks**	**Awarded**
Observation	**50**	
Record	**10**	
Total	**60**	

Result:

Thus, the program to develop a native calculator application has been executed successfully.

Ex No:4 Date:	DEVELOP A GAMING APPLICATION THAT USES 2-D ANIMATIONS AND GESTURES

Aim:

To develop a gaming application that uses 2-D animations and gestures.

Procedure:

1) Install Android Studio.

2) Set up a new project using Kotlin and integrate LibGDX.

3) Create 2D sprites for characters, backgrounds, and other elements using graphic design tools.

4) Animate characters using LibGDX's animation support.

5) Utilize LibGDX's gesture detection or implement a custom gesture recognition system.

6) Implement controls that respond to gestures for player navigation and interaction.

7) Define core game mechanics, challenges, and rewards, integrating gestures into gameplay.

8) Create different levels with increasing difficulty and design environments that encourage gesture-based interactions.

9) Add background music, sound effects, and any necessary audio assets.

10) Regularly test the game on Android devices to identify and fix bugs. Ensure smooth gesture-based interactions.

11) Optimize the game for performance, considering the limitations of mobile devices.

12) Prepare the game for release by configuring Android-specific settings.

13) Publish the game on the Google Play Store.

Program:

Main Game Class:

```
import com.badlogic.gdx.ApplicationAdapter
```

```
import com.badlogic.gdx.Gdx

import com.badlogic.gdx.graphics.GL20

import com.badlogic.gdx.graphics.Texture

import com.badlogic.gdx.graphics.g2d.SpriteBatch

class GestureQuest: ApplicationAdapter() {

private lateinit var batch: SpriteBatch

private lateinit var img: Texture
override fun create() {

batch = SpriteBatch()

img = Texture("badlogic.jpg") // Replace with your own image file

Joverride fun render() {

Gdx.gl.glClearColor(1f, 1f, 1f, 1f)

Gdx.gl.glClear(GL20.GL_COLOR_BUFFER_BIT)

batch.begin()

batch.draw(img, Of, Of)

batch.end()

} override fun dispose() {

batch.dispose()

img.dispose()

}

}
```

Gesture Recognition:

```
import com.badlogic.gdx.Gdx
```

```
import com.badlogic.gdx.InputProcessor

class MyInputProcessor: Input Processor {

override fun touchDown (screenX: Int, screenY: Int, pointer: Int, button: Int): Boolean {

// Handle touch down event

return true

}

override fun keyDown(keycode: Int): Boolean {

// Handle key down event

return false

}
// Implement other InputProcessor methods as needed

override fun touchUp (screenX: Int, screenY: Int, pointer: Int, button: Int) {}

override fun keyUp (keycode: Int) {}

override fun keyTyped(character: Char) = false

override fun touch Dragged(screenX: Int, screenY: Int, pointer: Int) {}

override fun mouseMoved (screenX: Int, screenY: Int) {}

override fun scrolled (amount: Int) {}

}
```

To use this input processor in your main game class, add the following code in the create method:

```
val inputProcessor = MyInputProcessor()

Gdx.input.inputProcessor = inputProcessor
```

Output:



EXPERIMENT(60 Marks)		
Description	**Max Marks**	**Awarded**
Observation	**50**	
Record	**10**	
Total	**60**	

Result:

Thus, the program to develop a gaming application that uses 2-D animations and gestures has been executed successfully.

Ex No:5 Date:	DEVELOP A MOVIE RATING APPLICATION (SIMILAR TO IMDB)

Aim:

To develop a movie rating application (similar to IMDB).

Procedure:

1) Install Java and Kotlin.

2) Set up your preferred IDE (IntelliJ IDEA is popular for Kotlin development).

3) Use the Spring Initializer to generate a new Spring Boot project with Kotlin and your preferred dependencies (Spring Web, Spring Data JPA, etc.).

4) Define entities for movies, users, reviews, etc.

5) Use Spring Data JPA to interact with the database.

6) Create controllers for handling HTTP requests (e.g., movie listing, user authentication, review submission).

7) Implement services to handle business logic.

8) Set up Spring Security for user authentication.

9) Integrate Spring Security for user registration, login, and profile management.

10) Allow users to submit and retrieve movie ratings.

11) Calculate and display average ratings.

12) Implement endpoints for users to submit and retrieve reviews.

13) Add a comment system for reviews.

14) Use a front-end framework (e.g., React, Angular, Vue) or templating engine (Thymeleaf) to create the user interface.

15) Implement pages for browsing movies, viewing details, and submitting reviews.

16) Implement search functionality using query parameters or a dedicated search endpoint.

17) Write unit tests and integration tests for your Kotlin code.

18) Use testing libraries like JUnit and Mockito.

19) Deploy your Spring Boot application to a platform like Heroku, AWS, or any other of your choice.

20) Implement security best practices, such as input validation and securing API endpoints.

21) Gather user feedback and consider adding new features or improving existing ones.

22) Implement monitoring tools to track application performance.

23) Regularly update dependencies and address security vulnerabilities.

Program:

```
data class Movie(val title: String, val genre: String, val release Year: Int)

data class Rating(val user: String, val movie: String, val rating: Double)

class MovieRatingSystem {

private val movies = mutableListOf<Movie>()
private val ratings = mutableListOf<Rating>()

fun addMovie(movie: Movie) {

movies.add(movie)

}

fun rateMovie(user: String, movie Title: String, rating: Double) {

val movie = movies.find { it.title == movieTitle}

if (movie != null) {

val userRating = Rating(user, movieTitle, rating)

ratings.add(userRating)

println("Rating added successfully.")
```

```
} else {
printin("Movie not found.")
}
}
fun getAverage Rating (movie Title: String): Double {
val movieRatings = ratings.filter {it.movie == movieTitle}
return if (movieRatings.isNotEmpty()) {
val totalRating = movieRatings.map {it.rating}.sum()
totalRating/movieRatings.size
} else {
0.0
}
}
fun listMovies() {
movies.forEach {
println("${it.title} (${it.releaseYear}) - ${it.genre}")
}
}
}
fun main() {
val movieRatingSystem = Movie RatingSystem()
// Adding movies
movieRatingSystem.addMovie (Movie("Inception", "Sci-Fi", 2010))
```

```
movieRatingSystem.addMovie (Movie("The Shawshank Redemption", "Drama", 1994))

movieRatingSystem.addMovie (Movie("The Dark Knight", "Action", 2008))

// Listing movies

println("Available Movies:")

movieRatingSystem.listMovies()

// Rating movies

movieRatingSystem.rateMovie("user1", "Inception", 9.0)

movieRatingSystem.rateMovie("user2", "Inception", 8.5)

movieRatingSystem.rateMovie("user1", "Black Widow", 9.5)

// Getting average ratings

println("Average Rating for Inception: ${movieRatingSystem.getAverage
Rating("Inception")}")

println("Average Rating for The Shawshank Redemption:
${movieRatingSystem.getAverageRating("The Shawshank Redemption")}")

println("Average Rating for The Dark Knight: ${movieRatingSystem.getAverage Rating("The
Dark Knight")}")

}
```

Output:





EXPERIMENT(60 Marks)		
Description	**Max Marks**	**Awarded**
Observation	**50**	
Record	**10**	
Total	**60**	

Result:

Thus, the program to develop a movie rating application (similar to IMDB) has been executed successfully.

Ex No:6 Date:	DEVELOP AN APPLICATION TO CONNECT TO A WEB SERVICE AND TO RETRIEVE DATA WITH HTTP

Aim:

To develop an application to connect to a web service and to retrieve data with HTTP.

Procedure:

1) Use your preferred IDE (such as IntelliJ IDEA) or a build tool (such as Gradle) to create a new Kotlin project.

2) If you're using Gradle, add the following dependencies to your build.gradle.kts (for Kotlin DSL) or build.gradle (for Groovy DSL): implementation "io.ktor:ktor-client-apache:$ktor_version" implementation "io.ktor:ktor-client-core:$ktor_version" implementation "io.ktor:ktor-client-serialization:$ktor_version"

3) Make sure to replace $ktor_version with the version of Ktor you want to use. 4) In your project, create a Kotlin file (e.g., Main.kt) to write your application code.

5) Write the code in 'Main.kt'

6) Use your IDE or build tool to run the Kotlin application. The program will make a GET request to the specified API URL, retrieve data, and print it to the console.

Program:

Main.kt

```
import io.ktor.client.HttpClient

import io.ktor.client.engine.apache.Apache

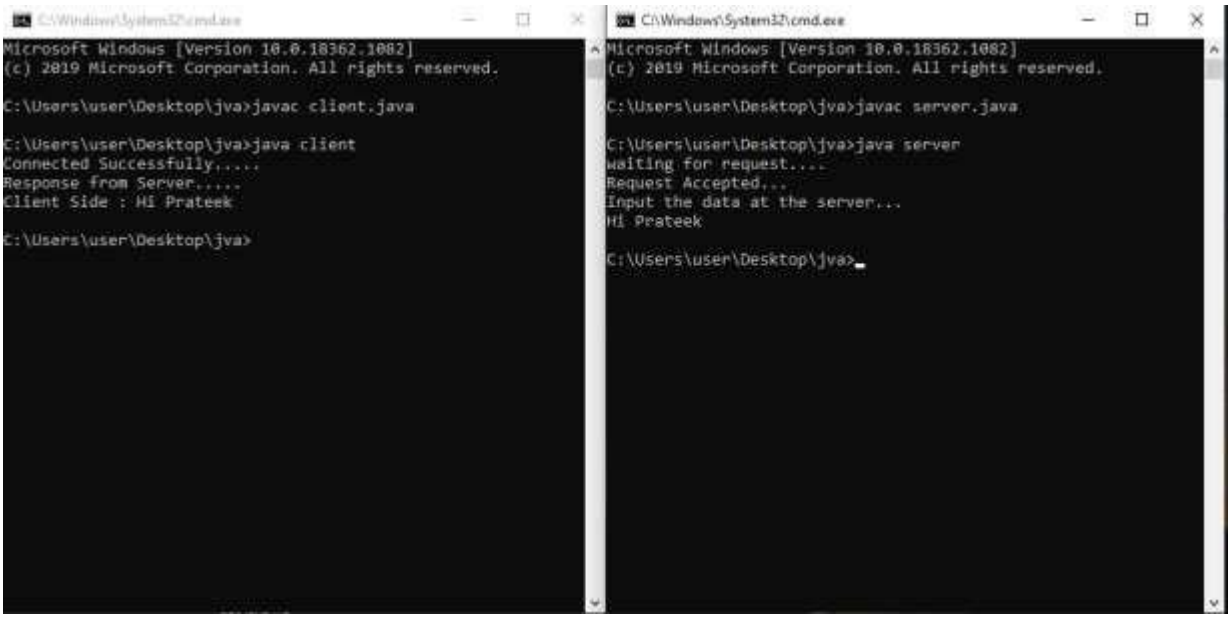
import io.ktor.client.request.get

suspend fun fetchData(url: String): String {

val client = HttpClient(Apache)
```

```
return try {
val response = client.get<String>(url)
response
} finally {
client.close()
}
}
fun main() {
val apiUrl = "https://api.example.com/data"
try {
val result = fetchData(apiUrl)
println("Data received: $result")
} catch (e: Exception) {
println("Error: ${e.message}")
}
}
```

Output:



EXPERIMENT(60 Marks)		
Description	**Max Marks**	**Awarded**
Observation	**50**	
Record	**10**	
Total	**60**	

Result:

Thus, the program to develop an application to connect to a web service and to retrieve data with HTTP has been executed successfully.

Ex No:7 Date:	DEVELOP A SIMPLE SHOPPING APPLICATION

Aim:

To Develop a simple shopping application.

Procedure:

1) Define the Product Data Class

2) Define Procedures for Displaying Products, Adding to Cart, and Viewing Cart

3) Implement the Main Shopping Cart Logic

4) Compile and run the Kotlin program

5) interact with the simple shopping app by choosing options from the menu.

Program:

```
import java.util.Scanner

data class Product(val id: Int, val name: String, val price: Double)

fun displayProducts (products: List<Product>) {

println("\nAvailable Products:")

for (product in products) {

println("${product.id). ${product.name} - $${product.price}")

}

}

fun addToCart(cart: MutableList<Product>, product: Product) {

cart.add(product)

println("${product.name} added to the cart.")
```

```
}
fun viewCart(cart: List<Product>) {
if (cart.isEmpty()) {
println("Your cart is empty.")
} else {
println("Your Cart:")
for ((index, item) in cart.withIndex()) {
println("${index + 1). ${item.name} - $${item.price}")
}
val totalPrice = cart.sumByDouble {it.price}
println("Total: $$totalPrice")
}
}
fun main() {
val scanner = Scanner(System. in')
val cart = mutableListOf<Product>()
val products = listOf(
Product(1, "Product A", 10.99), Product(2, "Product B", 19.99), Product(3, "Product C", 7.49)
println("Simple Shopping Cart!")
while (true) {
println("\nMenu:")
println("1. View Products")
```

```
println("2. Add to Cart")

println("3. View Cart")

println("4. Exit")

print("Enter your choice: ")

when (scanner.nextInt()) {

1->{
displayProducts(products)

}

2->{

println("\nEnter the product ID to add to cart:")

val productid = scanner.nextInt()

val selected Product = products.find { it.id == productid}

if (selected Product != null) {

addToCart(cart, selected Product)

} else {

println("Invalid product ID. Please try again.")

}

}

3-> {

viewCart(cart)

}

4->{

println("Thank you for using the Simple Shopping App. Exiting...")
```

```
return

}

else -> {

println("Invalid choice. Please enter a valid option.")

}

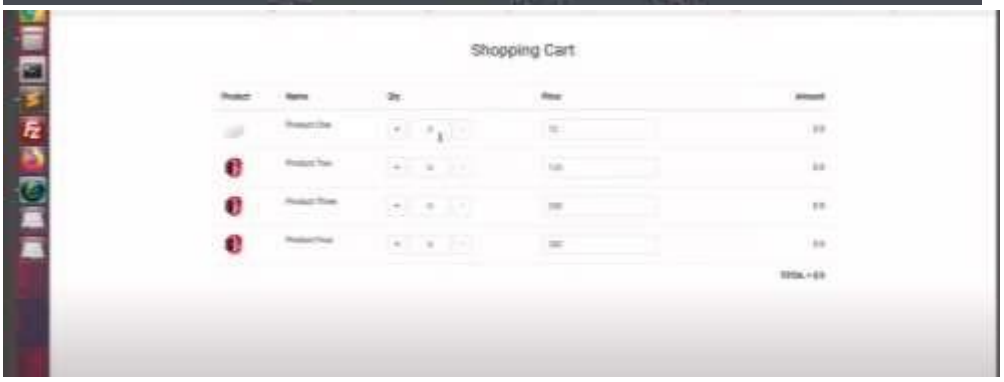
}

}

}
```

Output:





EXPERIMENT(60 Marks)		
Description	**Max Marks**	**Awarded**
Observation	**50**	
Record	**10**	
Total	**60**	

Result:

Thus, the program to develop a simple shopping application has been executed successfully.

Ex No:8 Date:	DESIGN A WEB SERVER SUPPORTING PUSH NOTIFICATIONS

Aim:

To design a web server supporting push notifications.

Procedure:

1) Create a new Kotlin project: Use your preferred IDE or build tool to create a new Kotlin project.

2) Add Ktor Dependencies: Open your build.gradle.kts (or build.gradle) file and include the following dependencies for Ktor:

```
plugins {

kotlin("jvm") version "1.5.31"

id("io.ktor") version "1.6.10"

}

repositories {

mavenCentral()

}

dependencies {

implementation("io.ktor:ktor-server-netty:1.6.10")

implementation("io.ktor:ktor-websockets:1.6.10")

}
```

1) Write Ktor Application Code: Create a new Kotlin file (e.g., Push NotificationServer.kt)

and write the Ktor application code:

2) Execute the main function in your Push NotificationServer.kt file. This will start the Ktor server on http://localhost:8080.

3) Create an HTML file (e.g., index.html) with a simple WebSocket client: Open the index.html file in a web browser. You can open multiple instances in different tabs or browsers to simulate multiple clients.

4) Interact with the WebSocket: Type a message in one client and click "Send Message." You should see the message pushed to all other connected clients in real-time.

Program:

Kotlin:

```
import io.ktor.application.*

import io.ktor.features.ContentNegotiation

import io.ktor.features.Status Pages

import io.ktor.http.HttpStatusCode

import io.ktor.jackson.jackson

import io.ktor.routing.routing

import io.ktor.server.engine.embeddedServer

import io.ktor.server.netty.Netty

import io.ktor.websocket.WebSockets

import io.ktor.websocket.webSocket

import kotlinx.coroutines.channels. Closed ReceiveChannelException

import kotlinx.coroutines.channels.consumeEach

import java.time.Duration

fun Application.module() {
```

```
install(ContentNegotiation) {

jackson {}

}

install(StatusPages) {

exception<Throwable> { cause ->

call.respondText(cause.localizedMessage, status = HttpStatusCode.Internal Server Error)

}

}

install(WebSockets) {

pingPeriod = Duration.ofSeconds(60)
timeout = Duration.ofSeconds(15)

maxFrameSize = Long.MAX_VALUE

masking = false

}

routing {

val connections = mutableListOf<Default WebSocketServerSession>()

webSocket("/push") {

connections.add(this)

try {

incoming.consumeEach {frame ->

if (frame is Frame.Text) {

val message = frame.readText()
```

```
connections.forEach {
if (it != this) {
it.send(Frame.Text(message))
}
}
}
}
} catch (e: Closed ReceiveChannelException) {
// Channel was closed normally
} finally {
connections.remove(this)
}
}
}
}
fun main() {
embeddedServer (Netty, port = 8080, module = Application::module).start(wait = true)
}
```

HTML file:

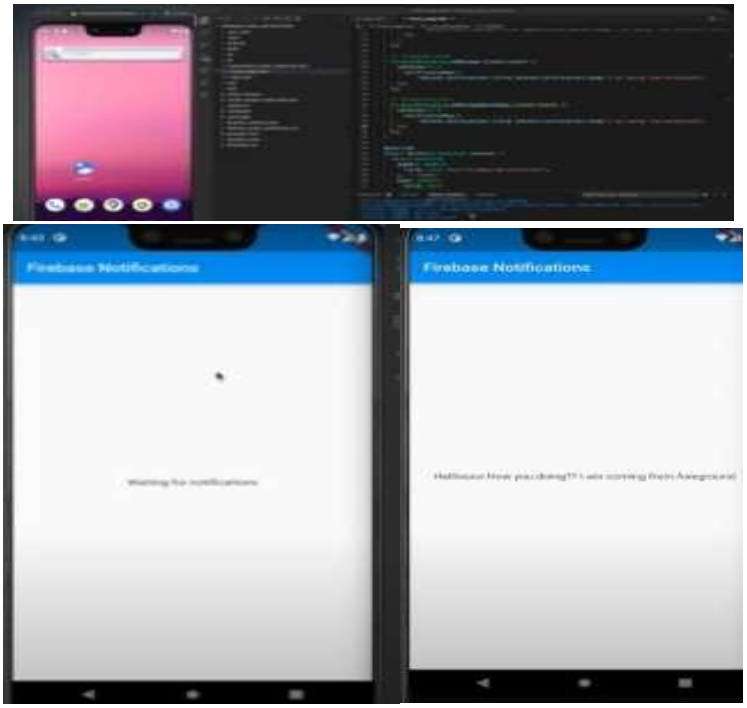
```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>WebSocket Push Notification</title>
</head>
<body>
<input type="text" id="messageInput" placeholder="Type a message">
<button onclick="sendMessage()">Send Message</button>
<ul id="messages"></ul>
<script>
const socket = new WebSocket("ws://localhost:8080/push");
socket.onmessage = (event) => {
const messages = document.getElementById("messages");
const li = document.createElement("li");
li.appendChild(document.createTextNode(event.data));
messages.appendChild(li);
};
function sendMessage() {
const input = document.getElementById("messageInput");
const message = input.value;
socket.send(message);
input.value = "";
}
</script>
```

</body>

</html>

Output:



EXPERIMENT(60 Marks)		
Description	**Max Marks**	**Awarded**
Observation	**50**	
Record	**10**	
Total	**60**	

Result:

Thus, the program to design a web server supporting push notifications has been executed successfully.

Ex No:9 Date:	DEVELOP AN APPLICATION BY INTEGRATING GOOGLE MAPS

Aim:

To develop an application by integrating Google maps

Procedure:

To integrate Google Maps into an Android application using Kotlin, you can follow these steps:

- Get API Key:
- Visit the Google Cloud Console.
- Create a new project or select an existing one.
- Enable the "Maps SDK for Android" for your project.
- Create an API key.
- Add Google Maps Dependency:
- Open your app-level build.gradle file and add the Google Maps dependency:
- Gradle implementation 'com.google.android.gms:play-services-maps:17.0.1'
- Add Permissions:

Make sure you have the necessary permissions in your AndroidManifest.xml file:

- xml

<uses-permission android:name="android.permission.INTERNET" />

<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />

<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />

- Add API Key:
- Add your Google Maps API key to the AndroidManifest.xml file:

XML Code:

```
<application>
<!-- Other application elements -->
<meta-data
android:name="com.google.android.geo.API_KEY"
android:value="YOUR_API_KEY" />
</application>
```

Create a Map Fragment:

Create a layout file (fragment_map.xml) for the map fragment:

```
<!-- fragment_map.xml -->
<fragment
android:id="@+id/mapFragment"
android:name="com.google.android.gms.maps.SupportMapFragment"
android:layout_width="match_parent"
android:layout_height="match_parent" />
```

Initialize GoogleMap:

In your activity or fragment, initialize the GoogleMap object:

Kotlin Program:

```
import com.google.android.gms.maps.GoogleMap
import com.google.android.gms.maps. OnMapReadyCallback
import com.google.android.gms.maps.SupportMapFragment
class MapFragment: Fragment(R.layout.fragment_map), OnMapReadyCallback {
```

```
private lateinit var googleMap: GoogleMap

override fun onViewCreated(view: View, savedInstanceState: Bundle?) {

super.onViewCreated(view, savedInstanceState)

val mapFragment = childFragmentManager.findFragmentById(R.id.mapFragment) as
SupportMapFragment

mapFragment.getMapAsync(this)

}
override fun onMapReady(map: GoogleMap) {

googleMap = map

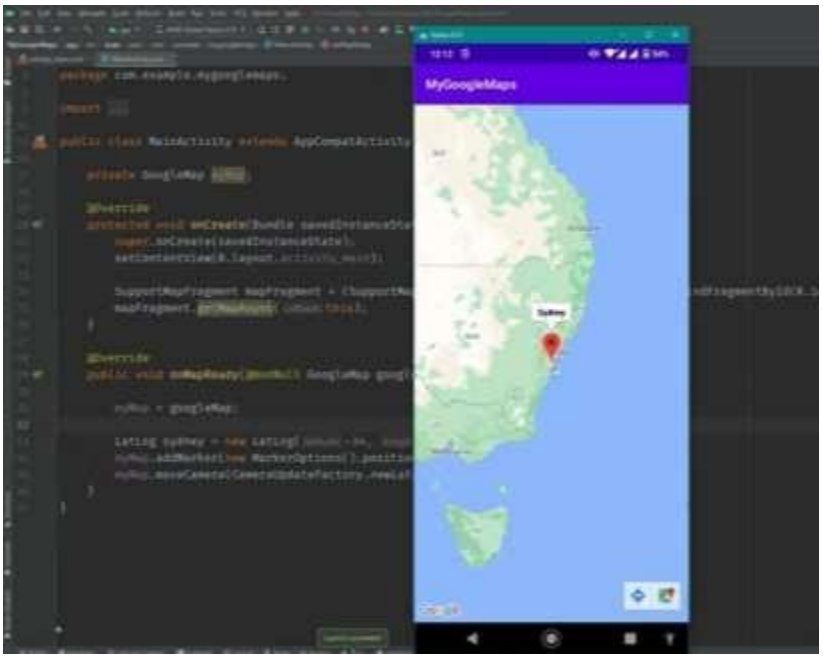
// Add your map configurations here

// For example: setMapType, addMarkers, etc.

}

}
```

Output:



EXPERIMENT(60 Marks)		
Description	**Max Marks**	**Awarded**
Observation	**50**	
Record	**10**	
Total	**60**	

Result:

Thus, the program to develop an application by integrating Google maps has been executed successfully.

Ex No:10 Date:	MINI PROJECT- BMI CALCULATOR APP

Aim:

To develop an application for calculation of BMI.

Procedure:

1. Set flutter bin path as the path in the environment variable. eg C:\flutter\bin
2. create a folder for workspace-flutter in any drive.
3. open command prompt and go to the same path. eg. E:\workspace-flutter
4. Enter the command "flutter create bmical", the app will be created in the location E:\workspace-flutter\bmical\
5. 5. open the vscode and open the project from the folder location eg. E:\workspace-flutter\bmical\
6. Choose terminal -> select new terminal
7. Enter the command "flutter run" – Choose chrome option among [1]windows, [2]chrome, [3]edge. Enter "2".
8. Chrome with load automatically, right click the page and select the inspect element.
9. Click "toggle device toolbar" -> select dimension in the screen top to navigate device and refresh the page.

```
import 'package:flutter/material.dart';

void main() => runApp(BMICalculator());

class BMICalculator extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: InputPage(),
    );
  }
}

class InputPage extends StatefulWidget {
  @override
  _InputPageState createState() => _InputPageState();
}

class _InputPageState extends State<InputPage> {
  @override
  Widget build(BuildContext context) {
```

```
    return Scaffold(
      appBar: AppBar(
        title: Text('BMI CALCULATOR'),
      ),
      body: Center(
        child: Text('Body Text'),
      ),
      floatingActionButton: FloatingActionButton(
        child: Icon(Icons.add),
      ),
    );
  }
}
```



EXPERIMENT(60 Marks)		
Description	**Max Marks**	**Awarded**
Observation	**50**	
Record	**10**	
Total	**60**	

Result:

Thus, the program to calculate Body Mass Index (BMI) has been executed successfully.

Ex No:11 Date:	CBS:REAL-TIME ON-DEVICE OBJECT DETECTION USING FLUTTER AND TENSORFLOW LITE

Aim:

To design, develop, and execute a cross-platform mobile application for real-time, on-device object detection using the Flutter framework integrated with a pre-trained TensorFlow Lite (TFLite) model.

Procedure:

1. Setup Flutter Project and Add Dependencies

- Create a new Flutter project by executing the following commands in the terminal:

  Bash

  ```
  flutter create object_detection_app
  cd object_detection_app
  ```

- Open the `pubspec.yaml` file and add the required dependencies for managing the camera hardware and handling the TensorFlow Lite inference engine:

  YAML

  ```
  dependencies:
    flutter:
      sdk: flutter
    camera: ^0.10.6
    tflite_flutter: ^0.11.0
  ```

- Run `flutter pub get` in the terminal to fetch and install the packages.

2. Import TFLite Model and Label Assets

- Download a mobile-optimized object detection model (e.g., `ssd_mobilenet.tflite`) along with its corresponding label text file (`ssd_mobilenet.txt`).
- Create an assets directory structure in your project root: `assets/models/`.
- Place your `.tflite` and `.txt` files inside that directory.
- Register the assets inside the `pubspec.yaml` file:

  YAML

  ```
  flutter:
    assets:
      - assets/models/ssd_mobilenet.tflite
      - assets/models/ssd_mobilenet.txt
  ```

3. Configure Platform-Specific Settings

- **Android Configuration:**

  Open `android/app/build.gradle` and ensure that the `.tflite` asset file is not compressed during the build process, allowing the interpreter to memory-map it directly:

  Groovy

```
android {
    ...
    aaptOptions {
        noCompress 'tflite'
    }
}
```

- **Permissions:** Ensure camera permissions are properly configured inside `AndroidManifest.xml` (for Android) and `Info.plist` (for iOS).

4. Load Model and Initialize Interpreter

- Implement an initialization function using `tflite_flutter` to load the model file from assets and configure interpreter options (such as threads or GPU hardware acceleration delegates):

  Dart

```
import 'package:tflite_flutter/tflite_flutter.dart';

late Interpreter interpreter;
List<String> labels = [];

Future<void> loadModel() async {
  final options = InterpreterOptions()..threads = 2;
  interpreter = await Interpreter.fromAsset(
    'assets/models/ssd_mobilenet.tflite',
    options: options
  );
  // Code to parse the ssd_mobilenet.txt lines into the labels list...
}
```

5. Process Live Camera Stream & Run Inference

- Initialize the `CameraController` using the `camera` package to stream live frame data.
- Pass the raw `CameraImage` bytes through a conversion layer to match the required format and input dimensions expected by the model (e.g., 300×300 pixels, normalized RGB matrix).
- Instantiate the input and output multidimensional matrices and process inference:

  Dart

```
// For an object detection model expecting a [1, 300, 300, 3] tensor
var input = [imageMatrix];
```

```
// Define structure for output bounding boxes, classes, and confidence
scores
var output = {
  0: List.filled(1, List.filled(10, List.filled(4, 0.0))), // Bounding
boxes
  1: List.filled(1, List.filled(10, 0.0)),                // Class IDs
  2: List.filled(1, List.filled(10, 0.0)),                 // Scores
  3: List.filled(1, 0.0)                                      // Total
Detections
};

interpreter.runForMultipleInputs([input], output);
```

- Filter results that pass a target confidence threshold (e.g., > 0.5) and render bounding boxes dynamically over the camera preview widget using a `CustomPainter` or a Stack of bounded `Positioned` widgets.

6. Build and Run the App

- Connect an Android/iOS device or start an emulator and run the application:

  Bash

  ```
  flutter run
  ```

EXPERIMENT(60 Marks)		
Description	**Max Marks**	**Awarded**
Observation	**50**	
Record	**10**	
Total	**60**	

Result:

Thus, the laboratory study and implementation of a real-time, on-device object detection mobile application using Flutter and TensorFlow Lite has been executed successfully.